From VP/DDIM to EDM (Images): A Minimal Spec for Multistep Consistency Distillation

Vinay Saji Mathew & Xiaowen You

January 26, 2026

Abstract

Diffusion models have achieved state-of-the-art results in image generation but suffer from slow sampling speeds due to the iterative nature of solving the Probability Flow ODE (PF-ODE). Consistency Models (CM) address this by learning to map any point on the ODE trajectory directly to the data distribution, enabling single-step generation. However, training Consistency Models from scratch is notoriously unstable. In this work, we propose a robust training framework that combines the *Elucidating the Design Space* (EDM) parameterization with the improved objectives of *Multistep Consistency Models*. We demonstrate that by utilizing EDM preconditioning and noise schedulesfor consistency targets, we can train a consistent model on ImageNet that balances the trade-off between sampling speed and generation quality.

1 Introduction

Generative modeling has seen a paradigm shift with the advent of Diffusion Probabilistic Models (DPMs). While models like Stable Diffusion achieve high fidelity, they typically require 20 to 100 function evaluations (NFEs) to generate a single image, effectively solving a reverse Stochastic Differential Equation (SDE) or Probability Flow ODE (PF-ODE). This computational bottleneck hinders real-time applications.

To address this, [1] introduced *Consistency Models*, which enforce a self-consistency property: points along the same PF-ODE trajectory should map to the same initial data point x_0. While theoretically enabling 1-step generation, the original training formulation suffers from high variance and biased targets due to the use of simple Euler solvers during training.

In this project, we synthesize three key advancements to train a high-quality consistency model on the ImageNet dataset:

1. **Architecture:** We adopt the robust preconditioning and architecture from the EDM framework [2].

2. **Objective:** We utilize the Multistep Consistency Loss [3], which reduces target bias.

Scope (images, UNet). This memo translates the common VP/DDIM formulation to the *EDM* parameterization used by image diffusion with a standard ADM/Dhariwal **UNet** (not UViT). It specifies: (i) EDM preconditioning, (ii) the probability–flow ODE, (iii) deterministic teacher stepping via *Heun*, (iv) the DDIM and inverse-DDIM formulas *in EDM space*, and (v) the **Multistep Consistency Distillation (MCD)** algrithm with S student steps. Everything here is for images normalized to $[-1, 1]$.

2 Background

2.1 The Probability Flow ODE

Diffusion models define a forward process that gradually adds noise to data x_0 to reach a prior distribution $x_T \sim \mathcal{N}(0, I)$. This process can be reversed by solving the Probability Flow ODE:

$$\frac{d\mathbf{x}}{dt} = f(t)\mathbf{x} + \frac{g^2(t)}{2\sigma(t)} \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \tag{1}$$

where $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$ is the score function learned by a neural network [4].

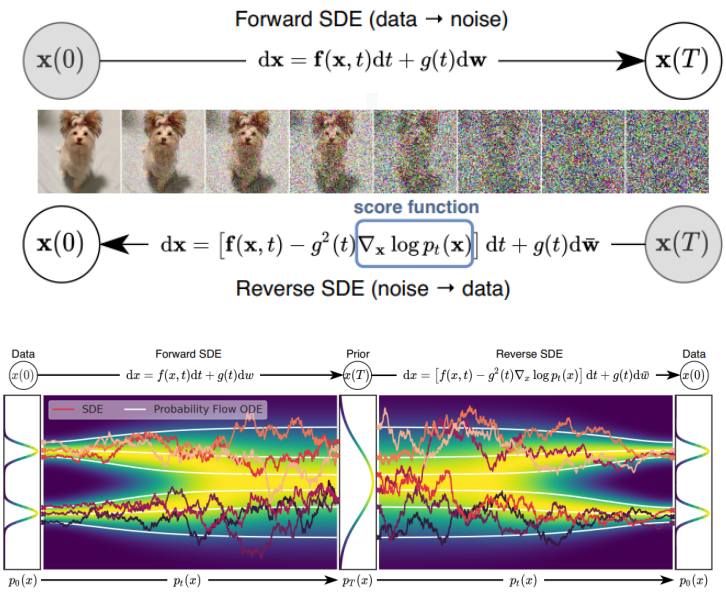


Figure 1: Diffusion Process for Images

2.2 EDM Parameterization: Elucidating the Design Space

Karras et al. (2022) [2] argue that the theory and practice of diffusion models have become unnecessarily convoluted. They propose a unified, modular framework that disentangles the design choices (SDE formulation, neural architecture, and sampler), allowing for independent optimization of each component. By "hot-swapping" these components, the authors demonstrate that improvements in one area (e.g., the sampler) can dramatically improve results for pre-trained models.

2.2.1 SDE Formulation: The Case for Zero Drift

Standard diffusion formulations often include complex drift terms (e.g., VP or VE schedules) that scale the data signal $s(t)$ during the forward process. EDM posits that these are unnecessary complications. Instead, the framework adopts a **zero drift** formulation where $s(t) = 1$ and the noise level is simply defined as $\sigma(t) = t$.

This simplification has a crucial geometric implication: the trajectories of the corresponding Probability Flow ODE become largely linear straight lines. Unlike the high-curvature paths of VP models, these linear trajectories are significantly easier for numerical solvers to integrate, reducing truncation error and improving sampling efficiency.

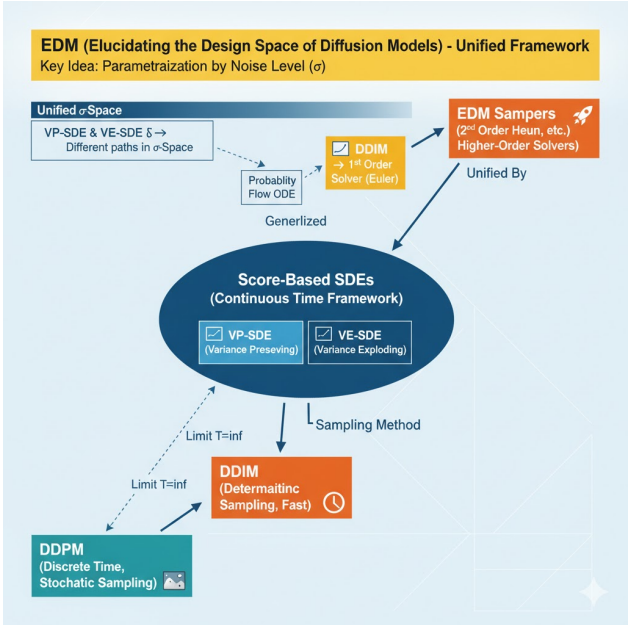


Figure 2: Relationships between different frameworks of diffusion models

2.2.2 Network Preconditioning

A central challenge in diffusion training is the vast difference in signal variance between high-noise (σ_{max}) and low-noise (σ_{min}) levels. To address this, EDM introduces a principled **preconditioning** scheme. Rather than predicting the score or noise directly, the network F_θ is wrapped in scaling coefficients to ensure unit variance for both inputs and training targets at all noise levels:

$$D_\theta(\mathbf{x};\sigma) = c_{\text{skip}}(\sigma)\mathbf{x} + c_{\text{out}}(\sigma)F_\theta(c_{\text{in}}(\sigma)\mathbf{x}; c_{\text{noise}}(\sigma)) \tag{2}$$

The coefficients $(c_{skip}, c_{out}, c_{in})$ are derived to minimize the variance of the effective training target, stabilizing the optimization landscape and preventing gradient explosions.

2.2.3 Optimized Training Dynamics

Leveraging this stable architecture, the authors introduce three key improvements to the training process:

- **Log-Normal Noise Distribution:** Instead of sampling training noise levels uniformly, EDM samples $\ln(\sigma)$ from a Log-Normal distribution ($P_{mean} = -1.2, P_{std} = 1.2$). This focuses network capacity on the perceptually critical "middle" noise range, avoiding wasted computation on indistinguishable high noise or trivial low noise.
- **Loss Weighting:** A geometric loss weighting $\lambda(\sigma)$ is applied to equalize the effective contribution of errors across different noise magnitudes.
- **Non-Leaking Augmentation:** To prevent overfitting on small datasets like CIFAR-10, the framework applies geometric augmentations (flips, rotations) but feeds the augmentation parameters as a condition to the network. This ensures the model learns the "clean" distribution without "leaking" augmentation artifacts into generated samples.

2.2.4 Deterministic Sampling and Stochasticity

Finally, the paper re-evaluates the role of stochasticity in sampling. While previous works (like DDPM) relied on stochastic noise injection to correct approximation errors, EDM finds that for a well-conditioned model, stochasticity is often "unnecessary but harmful." Instead, the authors propose a deterministic sampler called the aDDIM. However we stick to the **2nd-order Heun Sampler** as it is more well suited for the EDM scenario as described in Karras et al[2]. By combining the straight-line ODE trajectories (from the zero-drift formulation) with Heun's correction step, this sampler achieves a trade-off between image quality and step count (NFE) compared to Euler or ancestral samplers.

2.3 Consistency Models

While EDM optimizes the diffusion sampling process, it still fundamentally relies on iterative solvers (requiring $N \approx 20$ to 100 steps) to generate high-quality samples. [1] introduced Consistency Models to break this bottleneck, proposing a new family of generative models that support single-step generation by design.

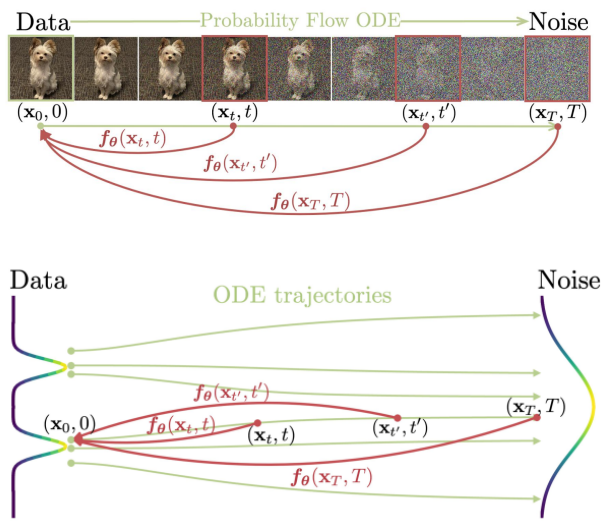


Figure 3: f_θ for Consistency Model

2.3.1 The Self-Consistency Property

The core idea is to learn a continuous function $f_\theta(\mathbf{x}_t, t)$ that maps any point $\mathbf{x}_t$ along the PF-ODE trajectory directly to its origin, the clean data $\mathbf{x}_0$. This function is defined by the *self-consistency property*: for any two time steps t and t' on the same trajectory, the model outputs must be identical:

$$f_\theta(\mathbf{x}_t, t) = f_\theta(\mathbf{x}_{t'}, t') = \mathbf{x}_0 \tag{3}$$

This property effectively distills the entire integration path into a single function evaluation. The model is trained by enforcing this consistency between adjacent time points, minimizing a metric $d(\cdot, \cdot)$ between the model's prediction at time t_{n+1} and a ··target" estimate derived from time t_n.

2.3.2 Limitations of the Original Formulation

Despite the theoretical promise of single-step generation, the original Consistency Models formulation suffers from several critical limitations that hinder their practical deployment:

1. **Biased Targets (Discretization Error):** The original training objective relies on a single first-order Euler step to estimate the target:

$$\hat{\mathbf{x}}_{t_n}^{\phi} \leftarrow \mathbf{x}_{t_{n+1}} + (t_n - t_{n+1}) f_{\theta^-}(\mathbf{x}_{t_{n+1}}, t_{n+1}) \tag{4}$$

   where θ^- represents a target network (exponential moving average of weights). This introduces significant discretization error (bias) that accumulates over the trajectory, causing the model to converge to a sub-optimal solution that produces blurry or inconsistent samples.

   Instead second order numerical solvers like Heun's methods are much more accurate, as is illustrated in Figure 4b.

2. **Training Instability:** The consistency loss is notoriously "hard to train." It suffers from high variance and optimization instability, often requiring careful hyperparameter tuning to prevent divergence.

3. **Trajectory constraints:** The model is forced to learn a mapping along a specific Probability Flow ODE trajectory determined by the noise schedule. If this trajectory is complex or high-curvature (which occurs with poor schedule choices), learning the direct mapping $f_\theta(\mathbf{x}_t, t) \to \mathbf{x}_0$ becomes exceedingly difficult.

4. **Sensitivity to Initialization:** The performance of consistency distillation is highly sensitive to the initialization of the student model. Without a well-trained teacher or a robust initialization strategy, the model often fails to capture the data distribution effectively.

2.3.3 Multistep Refinement

To address these instability and quality issues, [3] proposed **Multistep Consistency Models**. This framework improves the training objective in two critical ways:

1. **Extended Consistency Intervals:** Instead of enforcing consistency only between immediate neighbors t_{n+1} and t_n, the loss is computed over larger intervals $t_{n+k} \to t_n$. This reduces the total accumulation of discretization error over the trajectory.

2. **High-Order Target Estimation:** The simple Euler approximation is replaced with more accurate solvers during training. By using a **Heun solver** (or multiple solver steps) to compute the target $\hat{\mathbf{x}}_{t_n}^{\phi}$, the model learns from a far more accurate approximation of the true ODE trajectory.

3 Methodology: Hybrid Training Pipeline

Our approach integrates the EDM architecture with the Multistep Consistency training loop.

3.1 EDM parameterization (images)

We use a U-Net backbone wrapped in EDM preconditioning. The training noise schedule is sampled from a log-normal distribution:

$$\ln(\sigma_{\text{train}}) \sim \mathcal{N}(P_{mean}, P_{std}^2) \tag{5}$$

where $P_{mean} = -1.2$ and $P_{std} = 1.2$, as per [2]. The coefficients $c_{skip}, c_{out}, c_{in}$ are calculated dynamically based on the current batch's σ_{train}.

For inference, we work with noise level $\sigma > 0$ and a preconditioned denoiser

$$D_\theta(\boldsymbol{x};\sigma) \;=\; c_{\text{skip}}(\sigma)\,\boldsymbol{x} \;+\; c_{\text{out}}(\sigma)\,F_\theta\big(c_{\text{in}}(\sigma)\,\boldsymbol{x},\; c_{\text{noise}}(\sigma),\; \text{cond}\big), \tag{6}$$

where for images we typically set σ_{data}=0.5 and

$$c_{\text{in}} = \frac{1}{\sqrt{\sigma^2+\sigma_{\text{data}}^2}}, \quad c_{\text{skip}} = \frac{\sigma_{\text{data}}^2}{\sigma^2+\sigma_{\text{data}}^2}, \quad c_{\text{out}} = \frac{\sigma\,\sigma_{\text{data}}}{\sqrt{\sigma^2+\sigma_{\text{data}}^2}}, \quad c_{\text{noise}} = \tfrac{1}{4}\log\sigma. \tag{7}$$

The network predicts a denoised image in x-space via (6).

Destruction model (EDM form). EDM uses the simple additive corruption

$$\boldsymbol{x}_\sigma \;=\; \boldsymbol{x} + \sigma\,\boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(0, I). \tag{8}$$

3.1.1 VP/DDIM → EDM: the two key identities

VP uses $z_t = \alpha_t \boldsymbol{x} + \sigma_t \boldsymbol{\varepsilon}$ with $\alpha_t^2 + \sigma_t^2 = 1$. In EDM we stay in x-space with (8) and only pass σ to the network. The DDIM *and* inverse-DDIM maps in EDM are:

$$\textbf{DDIM}_{\text{EDM}}: \quad \boldsymbol{x}_{\sigma_s} = \hat{\boldsymbol{x}}_{\sigma_t} \;+\; \frac{\sigma_s}{\sigma_t}\big(\boldsymbol{x}_{\sigma_t} - \hat{\boldsymbol{x}}_{\sigma_t}\big), \qquad \hat{\boldsymbol{x}}_{\sigma_t} := D_\theta(\boldsymbol{x}_{\sigma_t};\sigma_t), \tag{9}$$

$$\textbf{invDDIM}_{\text{EDM}}: \quad \hat{\boldsymbol{x}}^\star_{\sigma_t} = \frac{\boldsymbol{x}_{\sigma_s} - \big(\frac{\sigma_s}{\sigma_t}\big)\,\boldsymbol{x}_{\sigma_t}}{1-\frac{\sigma_s}{\sigma_t}}. \tag{10}$$

These are the EDM–space analogues of the familiar VP/DDIM formulas and coincide with a single explicit PF-ODE Euler step when the step scale is 1.

3.1.2 Probability–flow ODE and samplers (deterministic)

With σ as time, the EDM PF-ODE is

$$\frac{d\boldsymbol{x}}{d\sigma} \;=\; \frac{\boldsymbol{x} - D_\theta(\boldsymbol{x};\sigma)}{\sigma}. \tag{11}$$

Euler (one eval at σ_a): stepping $\sigma_a \to \sigma_b$,

$$\boldsymbol{x}_{\sigma_b} \;=\; \underbrace{\frac{\sigma_b}{\sigma_a}}_{r}\,\boldsymbol{x}_{\sigma_a} \;+\; \big(1-r\big)\,\hat{\boldsymbol{x}}_{\sigma_a}, \qquad r = \frac{\sigma_b}{\sigma_a}, \tag{12}$$

which is exactly (9). **Heun (teacher; two evals)**:

$$\begin{aligned} k_1 &= \tfrac{\boldsymbol{x}_{\sigma_a} - \hat{\boldsymbol{x}}_{\sigma_a}}{\sigma_a}, \quad \boldsymbol{x}^{\text{eul}} = \boldsymbol{x}_{\sigma_a} + (\sigma_b - \sigma_a)\,k_1, \quad k_2 = \tfrac{\boldsymbol{x}^{\text{eul}} - \hat{\boldsymbol{x}}^{\text{eul}}_{\sigma_b}}{\sigma_b}, \\ \boldsymbol{x}^{\text{Heun}}_{\sigma_b} &= \boldsymbol{x}_{\sigma_a} + \tfrac{\sigma_b - \sigma_a}{2}\,(k_1 + k_2), \qquad \hat{\boldsymbol{x}}^{\text{eul}}_{\sigma_b} := D_\theta(\boldsymbol{x}^{\text{eul}};\sigma_b). \end{aligned} \tag{13}$$

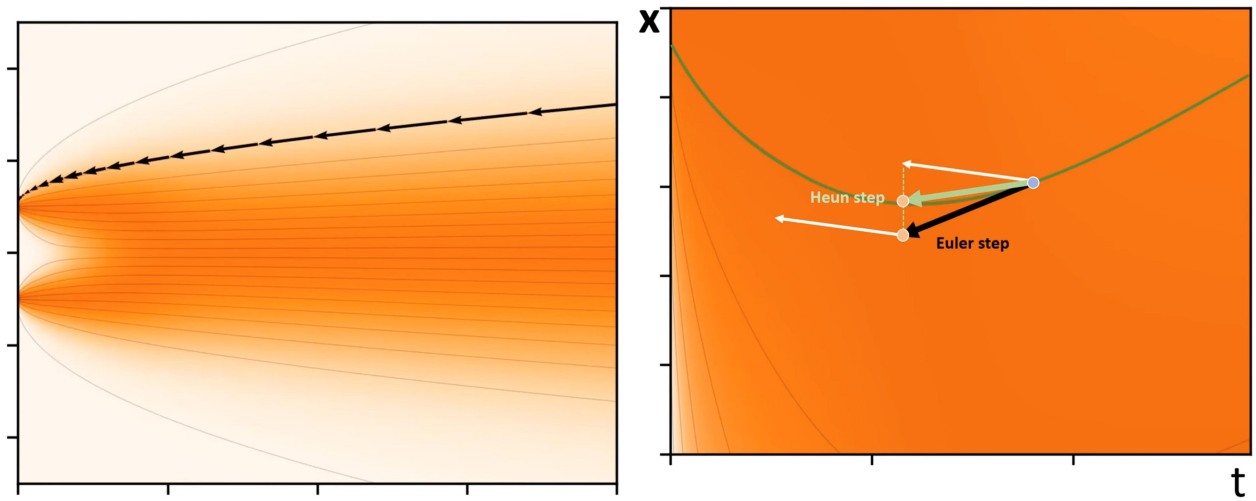
We keep the teacher deterministic (no churn).

3.1.3 Noise schedule (Karras style)

Use N descending noise levels $\{\sigma_k\}_{k=0}^{N}$ with a polynomial warp function

$$\sigma_k = \left(\sigma_{\max}^{1/\rho} + \tfrac{k}{N}\left(\sigma_{\min}^{1/\rho} - \sigma_{\max}^{1/\rho}\right)\right)^{\rho}, \quad k = 0, \ldots, N, \quad \rho \approx 7, \quad \sigma_{\min} \ll \sigma_{\max}. \tag{14}$$

For images: $\sigma_{\min} \approx 2\times 10^{-3}$, $\sigma_{\max} \approx 80$.



(a) Warp Function (b) Comparison between Euler and Huen Steps

Figure 4: Illustration of the path of noise schedule and data, RHS is noice distribution while LHS is data distribution

Figure 4a illustrates the path of noise using the warp function, LHS is data distribution while RHS is the noise distribution.

3.2 Multistep Consistency Distillation (images, UNet)

We distill a **UNet–EDM** student that runs in S steps from a stronger teacher integrated on a finer grid of T edges ($T \gg S$). All evaluations are in *EDM space* with (6).

Grids and segments

- **Teacher grid:** $\sigma_0^{\text{teach}} > \cdots > \sigma_{T-1}^{\text{teach}} > \sigma_T^{\text{teach}} = 0$ (using (14) and then append 0).
- **Student grid:** $\sigma_0^{\text{stud}} > \cdots > \sigma_{S-1}^{\text{stud}} > \sigma_S^{\text{stud}} = 0$.
- **Partition teacher edges into S contiguous groups** by rounding $k_b(j) = \left\lfloor \frac{jT}{S} + \frac{1}{2} \right\rfloor$ ($j = 0..S$). Segment j uses edges $e \in [k_b(j), k_b(j+1)-1]$ and anchors at the *right* student boundary $\sigma_{\text{bdry}}(j) := \sigma_{j+1}^{\text{stud}}$ (the last one is 0).

One training draw (CD, deterministic teacher)

Given an image $\boldsymbol{x}$ and class label:

1. Pick segment $j \in \{0, \ldots, S-1\}$ and a teacher *adjacent* pair (σ_t, σ_s) inside it (so $\sigma_t > \sigma_s$).

2. Sample $\boldsymbol{\varepsilon} \sim \mathcal{N}(0, I)$ and form $\boldsymbol{x}_{\sigma_t} = \boldsymbol{x} + \sigma_t \boldsymbol{\varepsilon}$.

3. **Teacher hop (Heun)**: $\boldsymbol{x}_{\sigma_s}^{\text{teach}} = \text{Heun}(\boldsymbol{x}_{\sigma_t}, \sigma_t \to \sigma_s)$ via (13) (no grad).

4. **Push to segment boundary at s with a single student DDIM/Euler:**

$$\boldsymbol{x}_{\text{bdry}}^{\text{ref}} \;=\; \hat{\boldsymbol{x}}_{\sigma_s}^{\text{ng}} \;+\; \frac{\sigma_{\text{bdry}}(j)}{\sigma_s}\Big(\boldsymbol{x}_{\sigma_s}^{\text{teach}} - \hat{\boldsymbol{x}}_{\sigma_s}^{\text{ng}}\Big), \qquad \hat{\boldsymbol{x}}_{\sigma_s}^{\text{ng}} := \text{stopgrad}\, D_\theta(\boldsymbol{x}_{\sigma_s}^{\text{teach}}; \sigma_s).$$

If $\sigma_{\text{bdry}}(j) = \sigma_s$, set $\boldsymbol{x}_{\text{bdry}}^{\text{ref}} := \boldsymbol{x}_{\sigma_s}^{\text{teach}}$.

5. **Invert at t to get the x-space denoiser target** (no network at σ=0):

$$\hat{\boldsymbol{x}}_{\sigma_t}^{\star} \;=\; \frac{\boldsymbol{x}_{\text{bdry}}^{\text{ref}} - \big(\frac{\sigma_{\text{bdry}}(j)}{\sigma_t}\big)\,\boldsymbol{x}_{\sigma_t}}{1 - \frac{\sigma_{\text{bdry}}(j)}{\sigma_t}} \quad \Big(\text{use (10); drop sample if } \sigma_{\text{bdry}}(j) = \sigma_t\Big).$$

6. **Student prediction and loss at t:**

$$\hat{\boldsymbol{x}}_{\sigma_t} \;=\; D_\theta(\boldsymbol{x}_{\sigma_t}; \sigma_t), \qquad \mathcal{L}(\sigma_t) \;=\; \lambda(\sigma_t) \cdot \rho\big(\hat{\boldsymbol{x}}_{\sigma_t} - \text{stopgrad}\, \hat{\boldsymbol{x}}_{\sigma_t}^{\star}\big).$$

Use Huber or ℓ_2; the *EDM weight*

$$\lambda(\sigma) \;=\; \frac{\sigma^2 + \sigma_{\text{data}}^2}{(\sigma\, \sigma_{\text{data}})^2} \tag{15}$$

3.2.1 Inference (student, S steps)

Let $\sigma_0^{\text{stud}} > \cdots > \sigma_S^{\text{stud}} = 0$. Starting from $\boldsymbol{x}_{\sigma_0^{\text{stud}}} \sim \mathcal{N}(0, \sigma_0^{\text{stud}2} I)$:

$$\text{for } b = 0..S-1: \quad \hat{\boldsymbol{x}}_{\sigma_b} = D_\theta(\boldsymbol{x}_{\sigma_b}; \sigma_b), \quad \boldsymbol{x}_{\sigma_{b+1}} = \hat{\boldsymbol{x}}_{\sigma_b} + \frac{\sigma_{b+1}}{\sigma_b}\big(\boldsymbol{x}_{\sigma_b} - \hat{\boldsymbol{x}}_{\sigma_b}\big).$$

This is the S-step deterministic sampler (EDM/DDIM/Euler).

Avoiding degenerate solutions in EDM–MCD. As noted by [1], a purely self–consistency objective of the form $\|D_\theta(\boldsymbol{x}_{\sigma_t}; \sigma_t) - \text{nograd}\, D_\theta(\boldsymbol{x}_{\sigma_s}; \sigma_s)\|$ admits trivial degenerate solutions (e.g. always predicting a constant). In our EDM multistep consistency distillation this collapse is ruled out because each student segment is *anchored* to a teacher trajectory that depends non–trivially on the data. In particular, whenever the teaching point σ_s coincides with the right boundary of a student segment, we set $\boldsymbol{x}_{\text{bdry}}^{\text{ref}} := \boldsymbol{x}_{\sigma_s}^{\text{teach}}$ and define the target denoiser at level σ_t via the EDM inverse–DDIM map

$$\hat{\boldsymbol{x}}_{\sigma_t}^{\star} \;=\; \text{invDDIM}_{\text{EDM}}\big(\boldsymbol{x}_{\text{bdry}}^{\text{ref}}, \boldsymbol{x}_{\sigma_t}\big) \;=\; \frac{\boldsymbol{x}_{\text{bdry}}^{\text{ref}} - (\sigma_{\text{bdry}}/\sigma_t)\,\boldsymbol{x}_{\sigma_t}}{1 - \sigma_{\text{bdry}}/\sigma_t},$$

cf. Eq. (10). The resulting loss term

$$\mathcal{L}(\sigma_t) \;=\; \lambda(\sigma_t)\, \rho\big(D_\theta(\boldsymbol{x}_{\sigma_t}; \sigma_t) - \text{nograd}\, \hat{\boldsymbol{x}}_{\sigma_t}^{\star}\big)$$

therefore reduces, in the last segment where $\sigma_{\text{bdry}} \to 0$ and $\boldsymbol{x}_{\sigma_s}^{\text{teach}} \to \boldsymbol{x}$, to the standard EDM denoising objective $\lambda(\sigma_t)\, \rho\big(D_\theta(\boldsymbol{x}_{\sigma_t}; \sigma_t) - \boldsymbol{x}\big)$. Since $\boldsymbol{x}$ and the corruption noise are sampled from $p_{\text{data}}(\boldsymbol{x})\mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$, the target $\hat{\boldsymbol{x}}_{\sigma_t}^{\star}$ is a non–constant function of $\boldsymbol{x}_{\sigma_t}$ with support of non–zero measure, and any predictor that ignores its input (such as a constant or collapsed solution) incurs strictly positive expected loss on this subset. Consequently, the only global minimizer of the EDM–MCD objective is the non–degenerate denoiser that correctly inverts the probability–flow ODE at each σ_t, and the multistep consistency constraint cannot collapse to a trivial solution.

References

[1] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever, "Consistency models," 2023. [Online]. Available: https://arxiv.org/abs/2303.01469

[2] T. Karras, M. Aittala, T. Aila, and S. Laine, "Elucidating the design space of diffusion-based generative models," 2022. [Online]. Available: https://arxiv.org/abs/2206.00364

[3] J. Heek, E. Hoogeboom, and T. Salimans, "Multistep consistency models," 2024. [Online]. Available: https://arxiv.org/abs/2403.06807

[4] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," 2021. [Online]. Available: https://arxiv.org/abs/2011.13456